

Makoya — The Plan

Makoya

June 23, 2026

CONTENTS

title: "Makoya — The Plan" subtitle: "Honest accessibility platform · strategy, status, roadmap & timeline" date: "June 2026"

1. What Makoya is (in one breath)
2. The strategic decision (and why it's the right one)
3. The four founder decisions (already made)
4. Where we are today (honest status)
5. How the product works — three points of view
6. The roadmap & timeline
- Demo phase (Weeks 1–2) — the loop that sells
- V1 (Weeks 3–6) — real customers
- V2 (later) — scale & differentiation
- Enterprise (when demand appears)
7. Architecture & tech stack
8. What I need from you (access & keys)
9. The demo: what's real vs mocked
10. Risks & how we handle them
11. Immediate next steps (in order)

title: “Makoya — The Plan” subtitle: “Honest accessibility platform · strategy, status, roadmap & timeline” date: “June 2026”

1. What Makoya is (in one breath)

Makoya sells and runs an **honest web-accessibility platform** for small and mid-size businesses. Three surfaces, one engine:

1. **Widget** — a `<script>` a client drops on their site. Gives visitors real usability preferences (text size, contrast, spacing, reading tools, read-aloud — 15 in total, 4 languages, 9 quick-profiles).
2. **Scanner** — a real WCAG 2.0/2.1/2.2 engine (Playwright + axe-core) that finds genuine accessibility problems and explains them in plain English. This is both the product's core value and the top of our sales funnel.
3. **Dashboard + Admin** — clients customize their widget and read their scan report; we (the operator) see every customer and lead, sorted worst-first, in an admin CRM.

The bet: the cheap “one line of code = you’re compliant” overlay market is collapsing under regulators and lawsuits. We win by being the honest alternative: find the real problems, help fix them at the source, monitor them over time, and offer a *useful* widget that never lies about what it does.

\newpage

2. The strategic decision (and why it’s the right one)

We researched the market deeply (AccessiBe, UserWay, AudioEye, EqualWeb, Level Access, Deque/axe, plus real user reviews and the accessibility community). The findings forced one clear decision.

The overlay-compliance model is being demolished from three directions:

- **Regulators:** The U.S. FTC fined **AccessiBe \$1,000,000** (final order April 2025) for falsely claiming its AI made sites WCAG-compliant — and separately for planting fake reviews. They are now *legally barred* from making those automated-compliance claims.
- **Courts:** Overlays don’t stop lawsuits. **22.6%** of all H1-2025 web-accessibility lawsuits hit sites that *already had an overlay installed*. AccessiBe appeared in 258 suits in 2024; UserWay in 187.
- **The users themselves:** A WebAIM survey found **67%** of accessibility practitioners — and **72%** of respondents *with disabilities* — rate overlays “not at all / not very effective.” Only 2.4% call them “very effective.” Blind users actively block these widgets.

Our position (locked): *Honest hybrid.* Real scanning + remediation guidance + monitoring + a preferences widget positioned as a genuine usability convenience — never as a “fix” or a compliance guarantee. We already accidentally built the right foundation (a real axe-core scanner, and our own rules forbid compliance claims). We lean in.

What we copy / improve / ignore:

From competitors	Our move
AccessiBe’s free scanner-as-lead-magnet	Copy the funnel, fix the dishonesty — show a real score + real issues, not a binary “NOT COMPLIANT” scare screen
AudioEye’s hybrid (automation + real human auditors)	Emulate the model, undercut the price
UserWay’s page-count scan pricing	Borrow the pricing structure
Deque’s axe Monitor (drift detection) + education-as-trust	Steal both
“Guaranteed compliance” claims, fake reviews, auto-detecting a user’s screen reader	Never build — these are the legal/ethical trap

\newpage

3. The four founder decisions (already made)

Decision	Choice
Positioning	Honest hybrid (not overlay-compliance)
Customer	SMB self-serve — product-led growth via the public scanner funnel
Budget	Bootstrap / open-source-first — under \$50/month + usage until revenue justifies more
Goal	Demo-first (a convincing end-to-end loop in ~2 weeks), then harden for real paying customers

4. Where we are today (honest status)

The hard, expensive parts already exist and are deployed (live at makoya-gamma.vercel.app): the real scanner, the 15-feature widget, Supabase auth + multi-tenant security, the customizer dashboard, and the admin CRM.

Phase 0 — Foundation: ✅ **DONE & verified.**

- Fixed the worst latent bug: the widget-config “single source of truth” was secretly a hand-copied file that could drift. It’s now generated from one canonical source, with a CI test that fails the build if they ever diverge.
- Added a CI pipeline (typecheck + tests on every push), an observability seam (ready for error tracking + analytics), and rewrote the stale docs to match reality.

Phase 1 — Revenue loop: 💎 **BUILT (against a stubbed email), demo-visible end-to-end.**

- A `leads` database table (private, operator-only).
- An email “seam”: today a **stub** that records the report instead of sending it (so the loop is demonstrable *without* a paid email service). Swapping in the real provider (Resend) is a one-file change later.
- A public scanner page (`/scan`): paste a URL → real score + severity breakdown + top issues in plain English → enter email → lead is captured.
- A public scan API with mandatory SSRF security (can’t be abused to scan internal networks) and rate limiting.
- An admin “Leads” page showing every captured lead, worst-score first.
- An honesty guardrail: a test literally fails the build if anyone reintroduces “compliant / guaranteed / lawsuit-proof” copy.

The one thing blocking “live”: the `leads` table migration must be applied to the Supabase database. Until then the loop is verified by typecheck + automated tests but not exercised against the live database. (Also: the real browser scan only runs on the cloud server, not on a local Windows machine — that’s expected.)

\newpage

5. How the product works — three points of view

The website visitor (uses the widget): sees a small launcher, opens a clean panel, picks preferences or a profile (low-vision, dyslexia, ADHD...). The widget never auto-detects their assistive tech, never claims to fix the site, and is dismissed in one click. This is the honesty line competitors cross and we don't.

The customer / site owner (the buyer): finds us → scans their site → gets a real score and an emailed report → signs up → lands straight in a live widget customizer with preview → installs one line of code → later gets monitoring alerts when a deploy breaks something → is offered AI-assisted (human-confirmed) fixes.

The operator — us (admin CRM): opens the admin, sorts customers/leads worst-first, and the worst-score unpaid leads *are* the call list. The product generates its own qualified sales pipeline.

The developer (integrates it): pastes a stable one-line snippet once and never touches it again — we ship new widget features without them changing anything. Later: a WordPress plugin, a documented API, and a CI gate that fails a pull request when it introduces accessibility violations.

6. The roadmap & timeline

Two-week target is a **convincing manager demo**; then we harden for real customers. Estimates assume one focused builder (me) plus your approvals/keys.

Demo phase (Weeks 1–2) — the loop that sells

When	Deliverable	Needs from you
✓ Done	Foundation, CI, docs (Phase 0)	nothing
✓ Built	Scanner funnel → lead → admin (Phase 1, vs email stub)	nothing
Next	Apply Leads migration → loop goes live	Supabase access (see §8)
Next	Swap stub → Resend (real emails)	Resend key + a domain to verify
Next	PDF report export	nothing
Next	Billing checkout (Lemon Squeezy, test mode) + server-side plan gating	Lemon Squeezy account
Next	Demo polish : honest-hybrid landing copy, “book a call” button, a funnel analytics dashboard	Calendly link; PostHog project

Demo script for your manager: “Watch — I paste a URL into our public scanner, it returns a real score and the real issues, I enter an email to get the full report, and a lead instantly appears in our admin dashboard. From there they self-serve sign up, customize the widget, and we can upsell monitoring.”

V1 (Weeks 3–6) — real customers

- Move heavy/monitoring scans off the request path onto a durable queue (Inngest, generous free tier).
- Scheduled re-scans + “your score dropped” alert emails.
- AI remediation **suggestions** (alt-text, plain-language fixes, code snippets) — always human-confirmed, never silently auto-applied.
- First install wrapper: a **WordPress plugin** (the biggest SMB surface).

V2 (later) — scale & differentiation

- Agency / white-label multi-client portal.
- Productized human-audit service (staffed via a partner, not a payroll team).
- Deeper analytics.

Enterprise (when demand appears)

- VPAT / ACR document generation, SSO/SAML, audit logs, SLA monitoring, a CI/CD accessibility gate for dev teams.

\newpage

7. Architecture & tech stack

Keep (already in place and good): Next.js 15 / React 19 / Tailwind v4 / shadcn UI · Supabase (Postgres + Auth + row-level security) · the Playwright + axe-core scanner tuned for serverless · Vercel hosting.

Add as we go (all chosen for cheap/free tiers):

Need	Choice	Cheaper/OSS note
Email	Resend	AWS SES is cheaper at high volume later
Billing	Lemon Squeezy (Merchant-of-Record — handles global tax for us)	Stripe is the fallback if we want more control
Background jobs / monitoring queue	Inngest (durable, great free tier)	Trigger.dev or Upstash QStash
Error tracking	Sentry	free tier
Product analytics	PostHog (open-source)	free tier
AI features	Anthropic Claude API	Haiku for bulk, Sonnet/Opus for hard cases
PDF reports	@react-pdf/renderer	free

Design principle: the widget config flows one way (dashboard → CDN JSON → widget), both ends share one type definition, the private `leads` /scan data is only ever touched by the trusted server, and AI output is stored as *suggestions* a human confirms.

8. What I need from you (access & keys)

For	What	When
Apply DB migrations	Supabase MCP (recommended) or the DB connection string in <code>.env.local</code> or paste SQL in the dashboard	now (unblocks the live loop)
Real emails	Resend API key + a sending domain you can add DNS records to	demo phase
Billing	Lemon Squeezy account (test mode is fine for the demo)	demo phase
Analytics + errors	PostHog + Sentry projects (free)	demo polish
"Book a call"	a Calendly link	demo polish
AI features	Anthropic API key (<code>ANTHROPIC_API_KEY</code>)	V1 (not needed for the demo)

Total running cost through the demo and early V1: under \$50/month plus tiny usage. The AI features cost single-digit dollars/month at our scale.

9. The demo: what's real vs mocked

- **Must be real:** scanner, score, report, email capture, lead in CRM, billing checkout (test mode). *(All built; live once the migration is applied + Resend swapped.)*
- **Can be mocked/simulated for the demo:** AI remediation (canned examples), the human-audit service (a Calendly link), enterprise VPAT generation.

10. Risks & how we handle them

Risk	Mitigation
Legal/regulatory (overlay claims)	Honest-hybrid positioning + a build-failing test that blocks compliance-claim copy
Scanner abused to probe internal networks (SSRF)	Mandatory SSRF guard on the public scan API + rate limiting
Scanner won't scale to monitoring many pages	Move heavy scans to a durable queue in V1
Vendor lock-in / cost creep	Bootstrap, open-source-first, Merchant-of-Record billing, provider-agnostic email seam

11. Immediate next steps (in order)

1. **You:** connect the Supabase MCP (or give me the DB URL), so I can apply the `leads` migration and take the loop live.
2. **You:** create the Resend account + verify a sending domain (DNS has lag — start early).
3. **Me:** apply migration → swap stub → Resend → real end-to-end loop; then PDF export; then Lemon Squeezy checkout (test mode); then demo polish.
4. **Together:** rehearse the manager demo on the live funnel.

Everything in this document is tracked live in `docs/SESSION.md` (phase status) and `docs/MASTER_CHECKLIST.md` (piece-by-piece). The competitor research is in `docs/research/COMPETITIVE_TEARDOWN.md`.